BSG Institute

# Programa Certified Python Data Engineer

**Proyectos de Ingeniería de Datos**

Fundamentos y aplicaciones avanzadas con herramientas modernas

Sesión 1

www.bsginstitute.com

BSG Institute

# Andrés Felipe Rojas Parra

- Ingeniero electrónico con mas de 23 años de experiencia
- PGP Artificial Intelligence & Machine Leaning at Texas University – Austin, TX
- Master en Inteligencia Artificial y Big Data at IEBSchool – Barcelona, Spain
- Estudiante de PGP en Cloud Computing at Texas University - Austin, TX
- Texas Executive Education Program – McCombs School of Business – Decision Science and AI Immersion Program
- Especialización en MLOps de la Universidad de Duke
- Estudiante de la especialización Deep Learning for Healthcare de la Universidad de Illinois

andres.rojas@triskelss.com
@arojaspa
https://www.linkedin.com/in/arojaspa/
+573005906373

BSG Institute

# Introducción a la aplicación de proyectos reales de ingeniería de datos

# Objetivo de la sesión:

- Comprender **cómo se aterrizan los conceptos de ingeniería de datos en proyectos reales**, dominar el **diseño lógico de pipelines multi-fuente en Python** y dejar **listo el entorno cloud** para el resto del curso y el proyecto final.

# Contexto de la sesión

**¿Por qué esta sesión es crítica en un curso de Ingeniería de Datos?**

**1.1 La realidad profesional de la Ingeniería de Datos (visión enterprise)**

En entornos empresariales reales, la **Ingeniería de Datos no es un ejercicio de programación**, sino una disciplina de **diseño de sistemas distribuidos orientados a datos**, donde el código es solo una consecuencia de decisiones previas mucho más profundas.

Un error común en perfiles junior o intermedios es asumir que:

**“El proyecto comienza escribiendo scripts en Python”.**

En la práctica profesional, **el código es el último paso**, no el primero.
Antes de escribir una sola línea de Python, un Data Engineer senior debe responder preguntas como:

- ¿Cuál es el **caso de uso de negocio** que consume estos datos?
- ¿El pipeline es **batch, near-real-time o streaming**?
- ¿Cuál es el **SLA esperado** (latencia, disponibilidad)?
- ¿Qué volumen de datos se espera hoy y en 2–3 años?
- ¿Qué nivel de **consistencia, calidad y gobernanza** se requiere?
- ¿Qué impacto tienen las decisiones técnicas en **costos operativos**?

Esta sesión introduce formalmente al estudiante en ese **modelo mental profesional**.

# Contexto de la sesión

**1.2 Los proyectos reales no empiezan con código, sino con arquitectura**

**Arquitectura como contrato técnico**

En proyectos reales, la arquitectura:

- Define **qué se construye**
- Delimita **qué no se construye**
- Condiciona costos, escalabilidad y mantenibilidad
- Sirve como contrato entre equipos (Data, ML, BI, Security, DevOps)

Antes del código, se define:

- Arquitectura lógica (capas, responsabilidades)
- Arquitectura física (servicios cloud concretos)
- Flujos de datos (data lineage)
- Puntos de fallo y recuperación
- Estrategias de escalamiento

# 1.2 Los proyectos reales no empiezan con código, sino con arquitectura

| Decisión | Impacto |
|---|---|
| Data Lake vs Data Warehouse | Costos, latencia, flexibilidad |
| Batch diario vs incremental | SLA, consumo de recursos |
| Push vs Pull ingestion | Control, resiliencia |
| ETL vs ELT | Performance, costos de compute |
| Almacenamiento columnar vs row-based | Query performance |

**Ejemplo de decisiones arquitectónicas tempranas, pensar en estas decisiones antes de escribir código**

# Contexto de la sesión

**1.3 Integración de múltiples fuentes heterogéneas: la norma, no la excepción**

En la industria, **el 100% de los proyectos reales integran múltiples fuentes**, y casi nunca son homogéneas.

Ejemplos típicos:
- APIs REST con límites de rate y autenticación
- Archivos CSV mal formateados
- Bases de datos legacy
- Datos semiestructurados (JSON anidados)
- Eventos generados por aplicaciones

Esto introduce problemas reales que no aparecen en ejercicios académicos simples:
- Diferentes esquemas y granularidades
- Diferentes frecuencias de actualización
- Inconsistencias semánticas
- Errores intermitentes de red
- Cambios de contrato en APIs

Por eso, desde esta primera sesión:
- Se abandona la idea de “un solo dataset limpio”
- Se introduce el concepto de **ingestión desacoplada**
- Se trabaja con **pipelines tolerantes al fallo**

# Contexto de la sesión

**1.4 Restricciones reales: costos, escalabilidad, observabilidad y seguridad**

Un pipeline que “funciona” técnicamente **no es suficiente** en un entorno empresarial.

**Costos:**

Cada decisión técnica tiene impacto económico:
- Tipo de almacenamiento
- Frecuencia de ejecución
- Volumen de datos procesados
- Servicios administrados vs custom

Un Data Engineer senior **optimiza por costo sin sacrificar SLA**.

**Escalabilidad:**

El pipeline debe:
- Funcionar con 1 GB hoy
- Seguir funcionando con 1 TB mañana
- No requerir reescritura completa

Esto implica:
- Diseño stateless
- Uso correcto de almacenamiento cloud
- Separación de compute y storage

# Contexto de la sesión

**1.4 Restricciones reales: costos, escalabilidad, observabilidad y seguridad**

**Observabilidad**

En producción, lo que no se observa no existe:

- Logs estructurados
- Métricas
- Alertas
- Trazabilidad de datos (data lineage)

Desde esta sesión se introduce:

- Logging como componente obligatorio
- Validaciones explícitas
- Manejo controlado de errores

**Seguridad**

Nunca se asume un entorno confiable:

- Principio de mínimo privilegio
- Gestión de credenciales
- Separación de entornos (dev/test/prod)
- Manejo seguro de secretos

# Contexto de la sesión

**1.5 La nube no es una opción, es el entorno natural**

Hoy, la Ingeniería de Datos moderna:

- **Vive en la nube**
- Escala horizontalmente
- Usa servicios administrados
- Se integra con ecosistemas analíticos y de ML

Por eso:

- El curso no se centra en ejecución local
- El entorno local es solo para desarrollo
- El estado final siempre está en cloud

El estudiante debe entender:

- Qué problema resuelve cada servicio cloud
- Qué responsabilidades delega a la plataforma
- Qué responsabilidades conserva como ingeniero

# Contexto de la sesión

**1.6 Rol pedagógico de esta sesión dentro del curso**

Esta sesión cumple tres funciones críticas:

**1. Puente entre teoría y práctica real**
Conecta:
- Conceptos aprendidos previamente
- Con escenarios empresariales reales
- Bajo restricciones reales

**2. Baseline técnico común**
Alinea a todos los estudiantes en:
- Nivel de herramientas
- Forma de pensar
- Estándares mínimos de calidad

Esto es clave en un curso de nivel maestría, donde:
- Los perfiles suelen ser heterogéneos
- El ritmo técnico es alto
- No hay tiempo para “nivelar” más adelante

**3. Setup obligatorio para evitar bloqueos futuros**
Muchos cursos fracasan no por el contenido, sino por:
- Entornos mal configurados
- Accesos cloud incompletos
- Falta de disciplina técnica inicial

Esta sesión **elimina esas fricciones desde el inicio**.

# Contexto de la sesión

**1.7 Resultado cognitivo esperado del estudiante**

Al finalizar este bloque contextual la idea es:

- Pensar primero en **arquitectura**, no en scripts
- Entender la ingeniería de datos como **sistema**, no como código aislado
- Reconocer que cada decisión técnica tiene consecuencias
- Estar mentalmente preparado para proyectos **de nivel empresarial**

# Revisión de herramientas y técnicas aprendidas previamente

# 2.1 Stack mínimo esperado

**Lenguaje y entorno**

- Python 3.10+
- Entornos virtuales (venv, pipenv o conda)
- Gestión de dependencias (requirements.txt)

**Manipulación de datos**

- pandas
- numpy
- Lectura de:
  - CSV
  - JSON
  - APIs REST
  - Bases de datos relacionales (intro)

**Conceptos ETL**

- Extract
- Transform
- Load
- Batch vs near-real-time
- Idempotencia básica

**Buenas prácticas**

- Modularización
- Separación de responsabilidades
- Manejo de errores
- Logging básico

BSG Institute

# Diseño de pipelines de datos con múltiples fuentes usando Python

# 3.1 Qué es un pipeline real

Un **pipeline de datos real**:

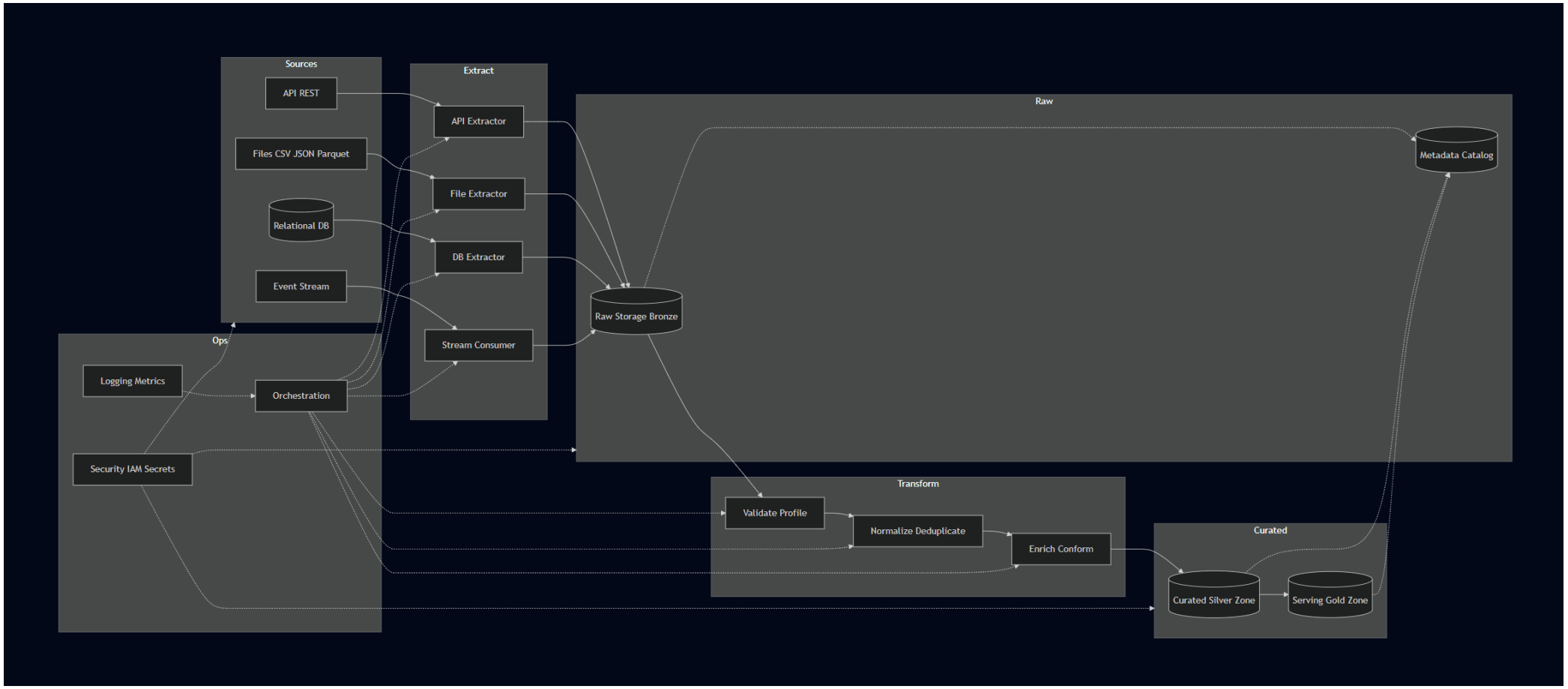
- Integra **más de una fuente**
- Aplica transformaciones reproducibles
- Es automatizable
- Tiene puntos de control y logging
- Puede escalar

# 3.2 Fuentes de datos típicas que se usarán en el curso

| Fuente | Tipo | Ejemplo |
|---|---|---|
| API pública | REST / JSON | Clima, finanzas, open data |
| Archivo | CSV/JSON | Ventas históricas |
| Base de datos | SQL | PostgreSQL / MySQL |
| Cloud storage | Objetos | S3 / GCS / Blob |

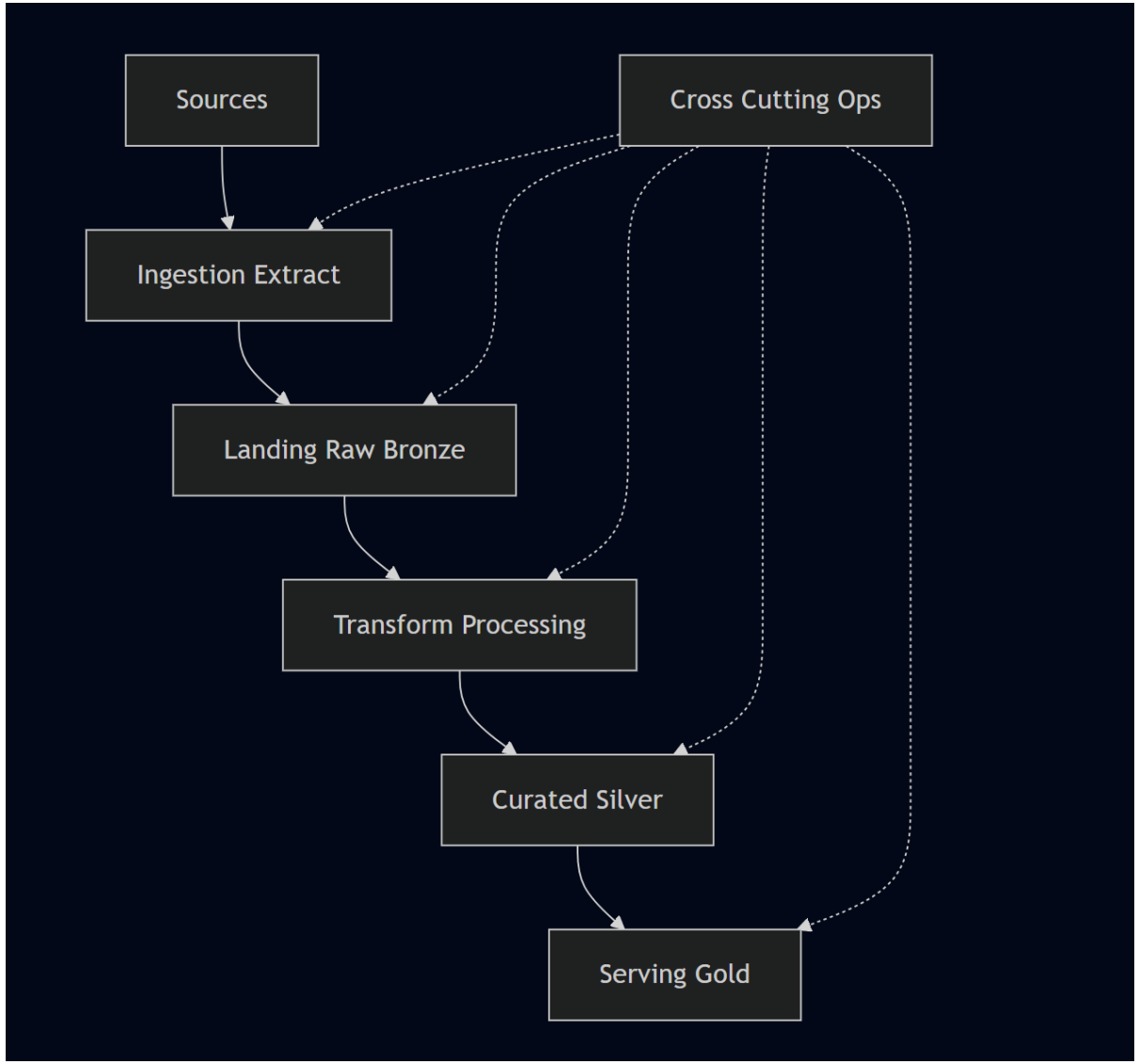
# 3.3 Diseño lógico del pipeline

Flujo principal (end-to-end, multi-fuente)



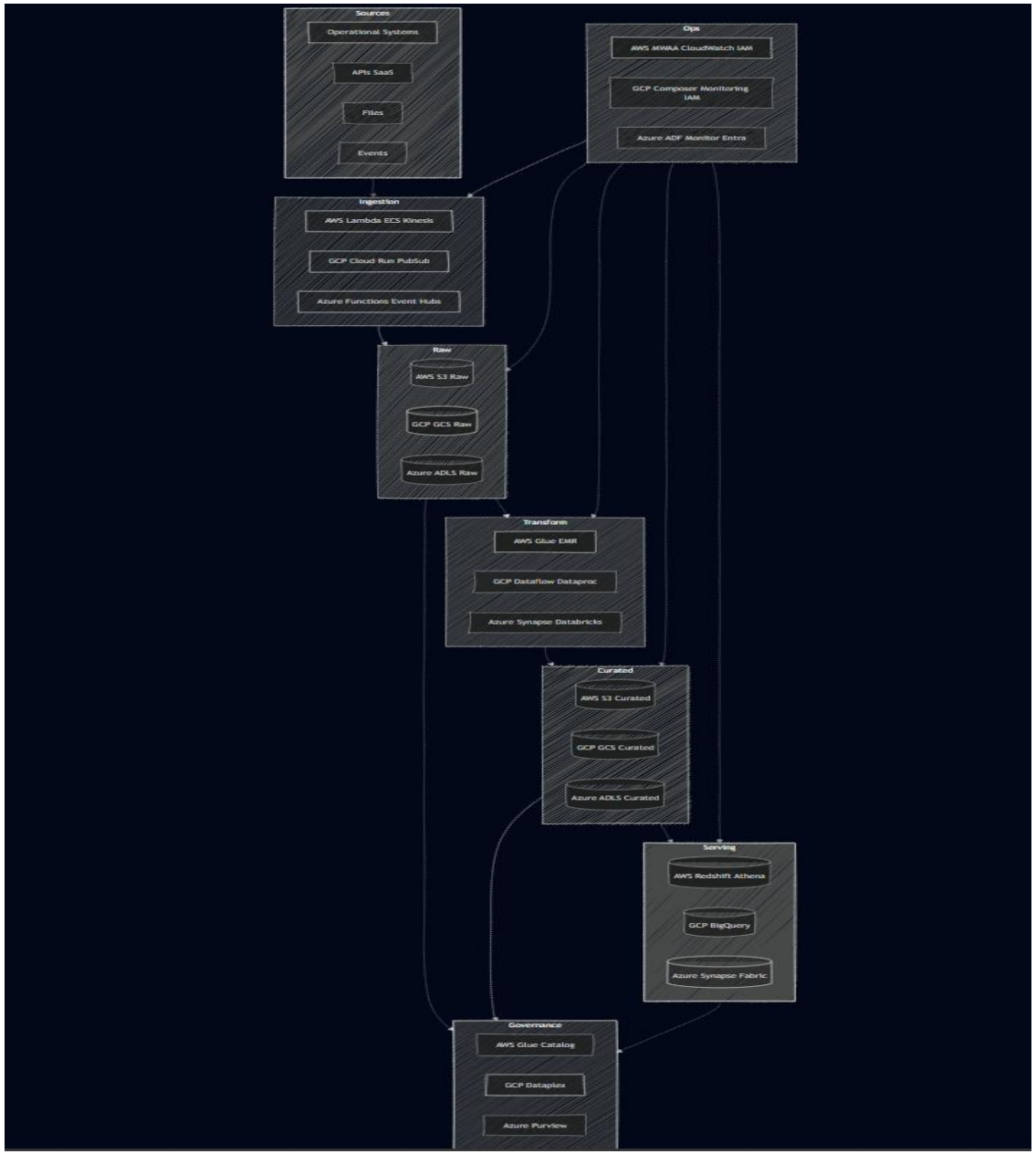
# 3.3 Diseño lógico del pipeline

**Arquitectura por capas**



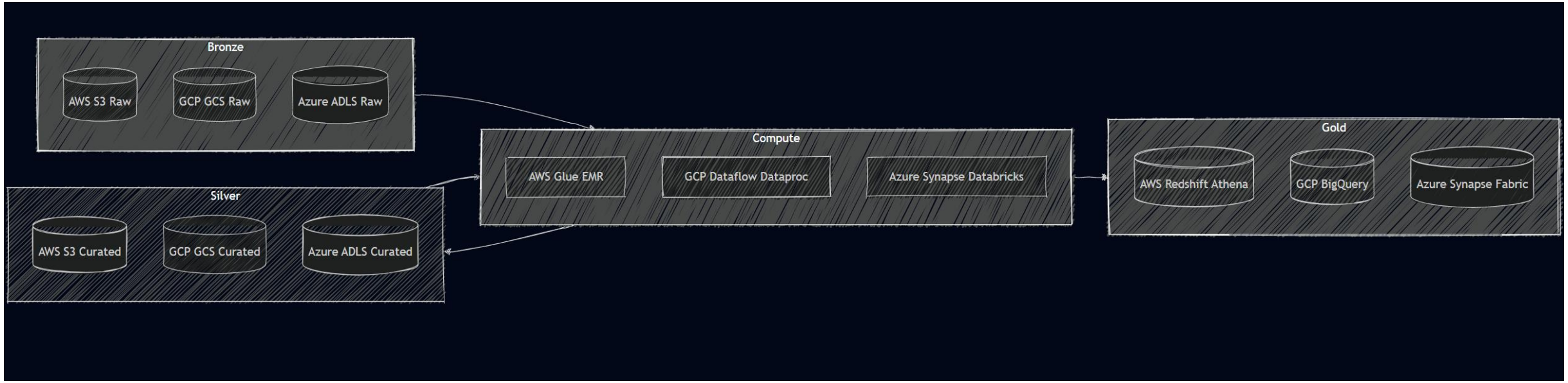
# 3.3 Diseño lógico del pipeline

Arquitectura multi-cloud con equivalencias (AWS GCP Azure)



# 3.3 Diseño lógico del pipeline

Medallion Lakehouse (Bronze Silver Gold)



# 3.4 Diseño técnico en Python

Estructura de proyecto recomendada:

```
data_pipeline_project/
├── src/
│   ├── extract/
│   │   ├── api_extractor.py
│   │   ├── csv_extractor.py
│   ├── transform/
│   │   ├── clean_data.py
│   │   ├── enrich_data.py
│   ├── load/
│   │   ├── load_s3.py
│   │   ├── load_gcs.py
│   ├── config/
│   │   └── settings.py
│   └── main.py
├── requirements.txt
├── README.md
└── .env
```

# 3.5 Principios de diseño que debes enfatizar

- Extract ≠ Transform ≠ Load
- Código desacoplado
- Configuración externa (.env)
- Reintentos y validaciones
- Preparación para orquestación futura (Airflow / Glue / Cloud Scheduler)

# Configuración inicial del entorno de trabajo en la nube

# 4.1 Enfoque del curso

- El estudiante **elige una nube**
- Explica las **equivalencias entre clouds**
- Evita vendor lock-in innecesario
- Se enfoca en **servicios core de Data Engineering**

# 4.2 Plataformas soportadas

**AMAZON WEB SERVICES**

**GOOGLE CLOUD PLATFORM**

**MICROSOFT AZURE**
(OPCIONAL, SI EL PROGRAMA LO PERMITE)

# 4.3 Setup mínimo obligatorio

- **Identidad y acceso**
- Cuenta cloud activa
- Usuario IAM (no root)
- Credenciales programáticas

# Servicios mínimos

| Capa | AWS | GCP |
|---|---|---|
| Storage | S3 | Cloud Storage |
| Compute (ligero) | EC2 / Lambda | Cloud Run / Functions |
| Data | Athena / RDS | BigQuery |

# Herramientas locales

AWS CLI / gcloud CLI

SDK Python (boto3, google-cloud-storage)

Variables de entorno

# 4.4 Validación técnica (hands-on)

**Cada estudiante debe:**

- Conectarse desde Python a la nube
- Crear un bucket
- Subir un archivo
- Listar objetos desde código

# 5. Cierre

**Entregables de la sesión**

- Entorno cloud funcional
- Estructura base del proyecto creada
- Conexión Python ↔ Cloud validada

**Preparación para la siguiente sesión**

- Dataset base descargado
- API seleccionada
- Repositorio Git inicializado

BSG Institute

Av. Alfredo Benavides 768, Miraflores – Lima
Urb. León XIII Calle 2 Nª 107, Cayma – Arequipa
Carrera 45 #108-27 Torre 1 Oficina 1008 – Bogotá
Av. Marcelo Terceros Bánzer 304 (3er anillo externo) – Santa Cruz
Montecito N° 38, Piso 33 of. 4 Col. Nápoles, World Trade Center – Ciudad de México

www.bsginstitute.com